

LLD – Notification Module (OpenScholar)

1. Overview

This document defines the Low-Level Design (LLD) for the Notification module of OpenScholar. The module is responsible for generating, storing, and delivering user notifications triggered by system events such as moderation decisions, comment replies, and other user interactions.

The design follows an event-driven approach with support for future real-time delivery.

2. Scope

The Notification module is responsible for:

- Create notifications based on system events
- Store notifications in database
- Retrieve user notifications
- Mark notifications as read/unread

Out of Scope:

- Email delivery system (can be integrated later)
 - Push notifications (future enhancement)
-

3. Data Model (Relevant Tables)

Tables used:

- notifications
- users

notifications

- id (UUID)
- user_id (UUID)
- type (VARCHAR)
- reference_id (UUID)
- message (TEXT)
- is_read (BOOLEAN)
- created_at (TIMESTAMP)

Indexes:

- INDEX on user_id
 - INDEX on is_read
-

4. Notification Types

Supported types:

- paper_approved
 - paper_rejected
 - comment_reply
 - system
-

5. API Contracts

5.1 Get User Notifications

Endpoint: GET /api/notifications

Headers: Authorization: Bearer <jwt>

Query Parameters:

- page (default: 1)
- limit (default: 10)

Response: { "total": 15, "results": [{ "id": "uuid", "type": "paper_approved", "message": "Your paper has been approved", "isRead": false, "createdAt": "timestamp" }] }

5.2 Mark Notification as Read

Endpoint: POST /api/notifications/read

Request Body: { "notificationId": "uuid" }

Response: { "message": "Notification marked as read" }

5.3 Mark All Notifications as Read

Endpoint: POST /api/notifications/read-all

Response: { "message": "All notifications marked as read" }

6. Event Triggers

Notifications are generated by other modules:

- Moderation Module:
 - paper approved → notify author
 - paper rejected → notify author
 - Engagement Module:
 - comment reply → notify original commenter
-

7. Service Logic

7.1 createNotification(userId, type, referenceId, message)

Steps: 1. Insert new notification into database 2. Return notification id

7.2 getUserNotifications(userId, page, limit)

Steps: 1. Query notifications by user_id 2. Apply pagination 3. Sort by created_at DESC 4. Return results

7.3 markAsRead(notificationId, userId)

Steps: 1. Validate notification belongs to user 2. Update is_read = true

7.4 markAllAsRead(userId)

Steps: 1. Update all notifications for user 2. Set is_read = true

8. Validation Rules

- user_id must exist
 - notification must belong to user for updates
 - type must be valid enum
-

9. Security Considerations

- Users can only access their own notifications
 - Validate JWT for all endpoints
-

10. Performance Considerations

- Index on user_id for fast lookup
 - Limit results per page
 - Avoid large payloads
-

11. Edge Cases

- No notifications found
 - Invalid notification ID
 - Multiple rapid events
-

12. Prisma Model Mapping

```
model Notification { id String @id @default(uuid()) userId String type String referenceId String message String isRead Boolean @default(false) createdAt DateTime @default(now()) }
```

13. Folder Structure

```
src/ app/api/notifications/route.ts app/api/notifications/read/route.ts app/api/notifications/read-all/route.ts  
  
modules/notification/service.ts modules/notification/repository.ts
```

14. Implementation Notes

- Keep notification creation lightweight
 - Avoid blocking main request flow
 - Use async triggers where possible
-

15. Acceptance Criteria

- User can view notifications

- User can mark notifications as read
 - Notifications are created on relevant events
 - Pagination works correctly
-

16. Future Enhancements

- Real-time notifications (WebSocket)
 - Email notifications
 - Push notifications
 - Notification preferences
-

End of Document